

INTRODUCTION

Le terme de « *Génie logiciel* » a été introduit à la fin des années soixante lors d'une conférence tenue pour discuter de ce que l'on appelait alors (*la crise du logiciel* « *software crisis* ») ... Le développement de logiciel était en crise. Les coûts du matériel chutaient alors que ceux du logiciel grimpaient en flèche. Il fallait de nouvelles techniques et de nouvelles méthodes pour contrôler la complexité inhérente aux grands systèmes logiciels. La crise du logiciel était perçue à travers ces symptômes :

- *La qualité du logiciel livré était souvent déficiente* : Le produit ne satisfaisait pas les besoins de l'utilisateur, il consommait plus de ressources que prévu et il était à l'origine de pannes.
- Les performances étaient très souvent médiocres (*temps de réponse trop lents*).
- Le non-respect des délais prévus pour le développement de logiciels ne satisfaisant pas leurs cahiers des charges.
- Les coûts de développement d'un logiciel étaient presque impossible à prévoir et étaient généralement prohibitifs (*excessifs*).
- L'invisibilité du logiciel, ce qui veut dire qu'on s'apercevait souvent que le logiciel développé ne correspondait pas à la demande (*on ne pouvait l'observer qu'en l'utilisant « trop tard »*).
- La maintenance du logiciel était difficile, coûteuse et souvent à l'origine de nouvelles erreurs.

Mais en pratique, il est indispensable d'adapter les logiciels car leurs environnements d'utilisation changent et les besoins des utilisateurs évoluent. Il est rare qu'on puisse réutiliser un logiciel existant ou un de ses composants pour confectionner un nouveau système, même si celui-ci comporte des fonctions similaires. Tous ces problèmes ont alors mené à l'émergence d'une discipline appelée « *le génie logiciel* ». Ainsi, Les outils de génie logiciel et les environnements de programmation pouvaient aider à faire face à ces problèmes à condition qu'ils soient eux-mêmes utilisés dans un cadre méthodologique bien défini.

Nul n'ignore que les plans logiciels sont connus pour *leurs pléthores de budget, et leurs cahiers des charges non respectés*. De façon générale, le développement de logiciel est une activité complexe, qui est loin de se réduire à la programmation. Le développement de logiciels, en particulier lorsque les programmes ont une certaine taille, nécessite d'adopter une méthode de développement, qui permet d'assister une ou plusieurs étapes du cycle de vie de logiciel.

Parmi les méthodes de développement, les approches objet, issues de la programmation objet, sont basées sur une modélisation du domaine d'application². Cette modélisation a pour but de faciliter la communication avec les utilisateurs, de réduire la complexité en proposant des vues à différents niveaux d'abstraction, de guider la construction du logiciel et de documenter les différents choix de conception.

Le génie logiciel (*software engineering*) représente l'application de principes d'ingénierie au domaine de la création de logiciels. Il consiste à identifier et à utiliser des méthodes, des pratiques et des outils permettant de maximiser les chances de réussite d'un projet logiciel. Il s'agit d'une science récente dont l'origine remonte aux années 1970. A cette époque, l'augmentation de la puissance matérielle a permis de réaliser des logiciels plus complexes mais souffrant de nouveaux défauts : délais non respectés, coûts de production et d'entretien élevés, manque de fiabilité et de performances. Cette tendance se poursuit encore aujourd'hui. L'apparition du génie logiciel est une réponse aux défis posés par la complexification des logiciels et de l'activité qui vise à les produire.

OBJECTS DU COURS

Ce cours a pour objectif principal, d'initier les étudiants, à la conception des applications informatiques de façon systématique (*méthodique*) et reproductible (*rééritable*); en les incitant à rechercher et établir les fonctionnalités d'une application, et à les modéliser sous forme de cas d'utilisation et de scénarios ainsi que rechercher les classes et les acteurs nécessaires à la conception de l'application. Et, D'une façon spécifique ce cours vise à :

- Acquérir aux étudiants qui auront suivi ce cours, les bonnes pratiques de conception, comme l'utilisation de patron de conception (*design pattern*), le découpage en couches de l'architecture, la structuration en paquetages et le maquetage ;
- Maîtriser des techniques de génie logiciel, en se focalisant sur les approches par objets et par composants ;
- Présenter un aperçu de l'état de l'art en matière de génie logiciel.
- Exposer les principaux courants de pensées en matière de développement logiciel.
- Proposer un ensemble de pratiques pragmatiques qui permettent de survivre à un projet de développement de logiciel.

DEFINITION DES CONCEPTS CLES

- **Le génie logiciel** est un domaine des sciences de l'ingénieur dont l'objet d'étude est la *conception*, la *fabrication*, et la *maintenance* des *systèmes informatiques complexes*.
- **Un système** est un ensemble d'*éléments* interagissant entre eux suivant un certains nombres de principes et de *règles* dans le but de réaliser un *objectif*.
- **Un logiciel**³ est un ensemble d'entités nécessaires au fonctionnement d'un processus de traitement automatique de l'information. Parmi ces entités, on trouve par exemple : des programmes (en format *code source* ou exécutables);des documentations d'utilisation ;des informations de configuration.
- **Un modèle** : est une représentation schématique de la réalité.
- **Une base de Données**: ensemble des données (*de l'organisation*) structurées et liées entre elles : stocké sur support à accès direct (*disque magnétique*) ; géré par un SGBD (*Système de Gestion de Bases de Données*), et accessible par un ensemble d'applications.
- **Une analyse** : c'est un processus d'examen de l'existant
- **Une Conception** : est un processus de définition de la future application informatique.
- **Un système d'Information** : ensemble des moyens (*humains et matériels*) et des méthodes se rapportant au traitement de l'information d'une organisation.

PREMIER CHAPITRE – GENERALITES SUR LE GENIE (L'INGENIERIE) LOGICIEL

Le génie logiciel est une branche de l'ingénierie associée au développement de logiciels utilisant des principes, méthodes et procédures scientifiques bien définis. Le résultat de l'ingénierie logicielle est un produit logiciel efficace et fiable. Commençons par comprendre ce que signifie « *le génie logiciel* ». Le terme est composé de deux mots, le logiciel et l'ingénierie :

- *Le logiciel* est plus qu'un code de programme. En faite, un programme est un code exécutable, qui sert à des fins de calcul, par contre, le logiciel, est considéré comme une collection de code de programmation exécutable, des bibliothèques associées et de documentations. Ainsi, lorsque le logiciel, est conçu pour une exigence spécifique, est appelé un « *produit logiciel* ».



- D'autre part, *l'ingénierie (génie)* consiste à développer des produits, en utilisant des principes et méthodes scientifiques bien définis.

II.1. DEFINITION ET CONTEXTE D'ETUDES

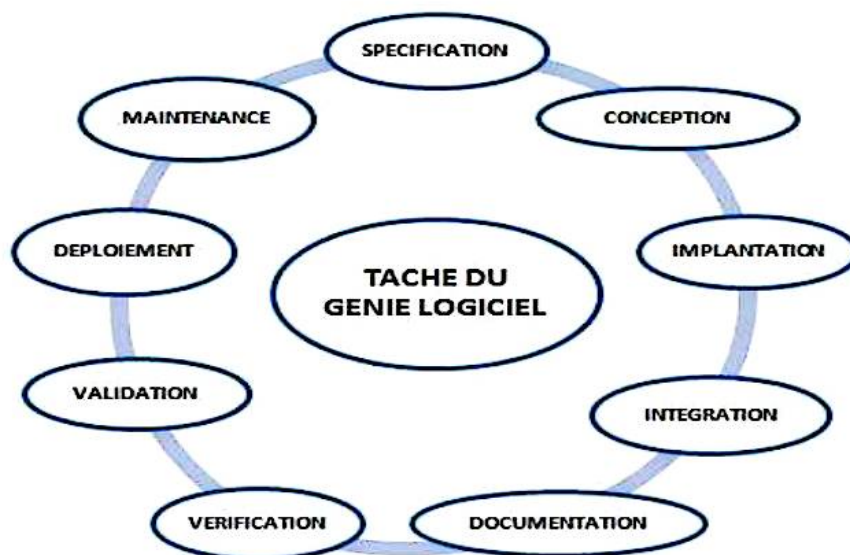
IEEE définit le génie logiciel comme l'application d'une approche systématique, disciplinée et quantifiable au développement, à l'exploitation et à la maintenance des logiciels; c'est-à-dire l'application de l'ingénierie aux logiciels.

Fritz Bauer, un informaticien allemand, définit le génie logiciel comme l'établissement et l'utilisation de principes d'ingénierie afin d'obtenir des logiciels économiques, fiables et efficaces sur des machines réelles.

Dans le cadre de ce cours, le génie logiciel⁴ sera considéré comme étant un ensemble d'activités de conception et de mise en œuvre des produits et des procédures tendant à rationaliser (*normaliser*) la production du logiciel et son suivi. Autrement dit, le génie logiciel est l'art de produire de bons logiciels, au meilleur rapport qualité prix. Remarquons que cette définition fait référence à deux aspects indispensables dans la conception d'un logiciel :

- *Le processus ou procédure de développement des logiciels* - ensemble de formalités, des marches à suivre et des démarches pour obtenir un résultat déterminé et ;
- *La maintenance et le suivi des logiciels* - ensemble d'opérations permettant de maintenir le fonctionnement d'un équipement informatique.

Le génie logiciel englobe les tâches suivantes :



- **La Spécification** : capture des besoins, cahier des charges, spécifications fonctionnelles et techniques

- **La Conception** : analyse, choix de la modélisation, définition de l'architecture, définition des modules et interfaces, définition des algorithmes
- **L'Implantation** : choix d'implantations, codage du logiciel dans un langage cible
- **L'Intégration** : assemblage des différentes parties du logiciel
- **La Documentation** : manuel d'utilisation, aide en ligne
- **La vérification** : tests fonctionnels, tests de la fiabilité, tests de la sûreté
- **La Validation** : recette du logiciel, conformité aux exigences du CDC
- **Le Déploiement** : livraison, installation, formation
- **La Maintenance** : corrections, et évolutions.

I.2. ÉVOLUTION DU LOGICIEL

Le processus de développement d'un logiciel à l'aide de principes et de méthodes du génie logiciel est appelé « *évolution logicielle* ». Cela inclut le développement initial du logiciel et sa maintenance et ses mises à jour, jusqu'à ce que le logiciel désiré soit développé, ce qui répond aux exigences attendues.

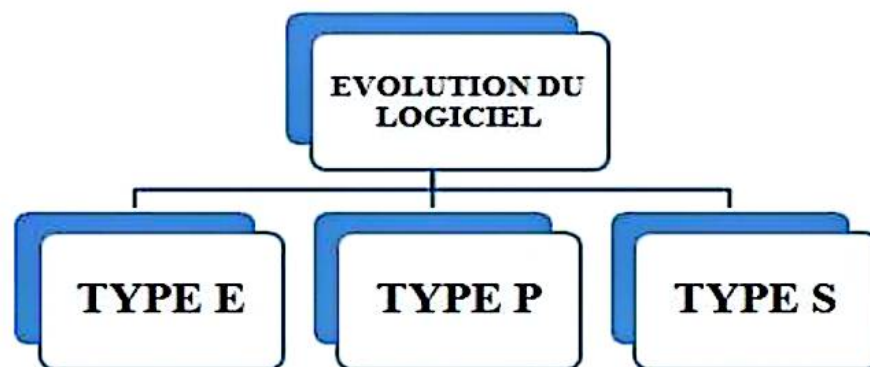


L'évolution commence par le processus de *collecte des exigences*. Après quoi les développeurs créent un *prototype* du logiciel prévu et le montrent aux utilisateurs pour obtenir leurs commentaires dès les premières étapes du développement de logiciels. Les utilisateurs suggèrent des modifications sur lesquelles plusieurs mises à jour et maintenances consécutives continuent à évoluer. Ce processus passe au logiciel d'origine, jusqu'à ce que le logiciel souhaité soit accompli.

Même lorsque l'utilisateur dispose du logiciel souhaité, la technologie évolutive et les exigences changeantes forcent le logiciel à changer en conséquence. Récréer des logiciels à partir de zéro et aller en tête-à-tête avec des exigences n'est pas réalisable. La seule solution possible et économique consiste à mettre à jour le logiciel existant afin qu'il corresponde aux dernières exigences.

I.2.1. LOIS D'EVOLUTION DU LOGICIEL

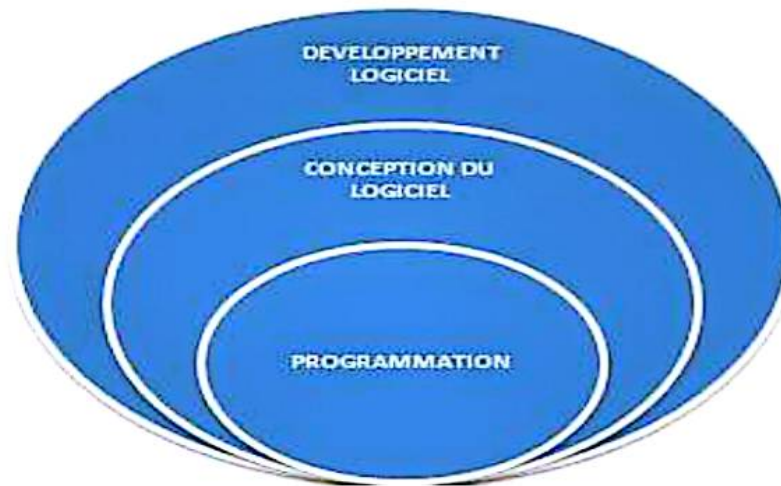
Meir Lehman a donné des lois sur l'évolution des logiciels. Il a divisé le logiciel en trois catégories différentes :



- *Type S (type statique)* - Il s'agit d'un logiciel qui fonctionne selon des spécifications et des solutions définies. La solution et la méthode pour y parvenir, les deux sont immédiatement comprises avant le codage. Le logiciel de type S est le moins soumis à des modifications, ce qui en fait le plus simple. Par exemple, programme de calculatrice pour le calcul mathématique.
- *Type P (type pratique)* - Il s'agit d'un logiciel avec un ensemble de procédures. Ceci est défini par exactement ce que les procédures peuvent faire. Dans ce logiciel, les spécifications peuvent être décrites mais la solution n'est pas évidente instantanément. Par exemple, un logiciel de jeu.
- *E-type (type embarqué)* - Ce logiciel fonctionne étroitement comme exigence d'un environnement réel. Ce logiciel a un haut degré d'évolution car il existe divers changements dans les lois, les taxes, etc. dans les situations réelles. Par exemple, logiciel de trafic en ligne.

I.3. LES PARADIGMES LOGICIELS

Les paradigmes logiciels font référence aux méthodes et aux étapes qui sont prises lors de la conception du logiciel. Il existe de nombreuses méthodes proposées et sont en cours de réalisation, mais nous avons besoin de voir où se situent ces paradigmes dans le génie logiciel. Ceux-ci peuvent être combinés en différentes catégories, bien que chacun d'eux soit contenu l'un dans l'autre :



Il est à noter que le paradigme de la programmation est un sous-ensemble du paradigme de conception de logiciels, qui est en outre un sous-ensemble du paradigme de développement logiciel.

I.3.1. PARADIGME DE DEVELOPPEMENT LOGICIEL

Ce paradigme est connu sous le nom de paradigmes d'ingénierie logicielle, où tous les concepts d'ingénierie relatifs au développement de logiciels sont appliqués. Il comprend diverses recherches et la collecte des exigences qui aident le produit logiciel à construire. Ce paradigme fait partie du développement logiciel et inclut : *Le rassemblement des besoins ; la Conception des logiciels et la planification des tâches.*

I.3.2. PARADIGME DE CONCEPTION

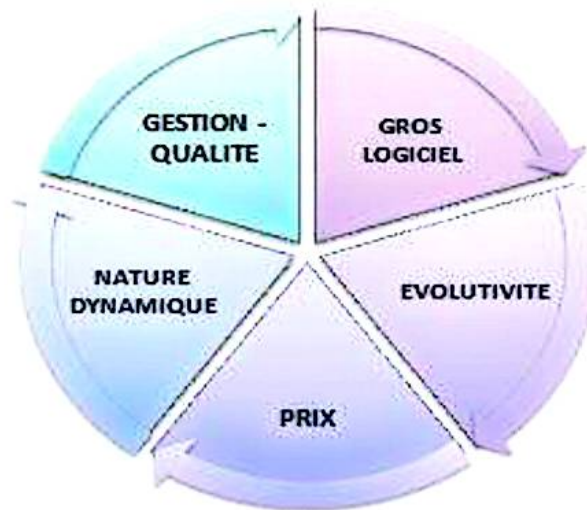
Ce paradigme fait partie du développement de logiciels et comprend : *la conception ; la maintenance et l'organisation des activités à exécuter.*

I.3.3. PARADIGME DE PROGRAMMATION

Ce paradigme est étroitement lié à l'aspect programmation du développement logiciel. Cela inclut : *Le codage ; Le Test et L'intégration des besoins.*

I.4. BESOINS DU GENIE LOGICIEL

Les besoins du génie logiciel est dû au taux de changement plus élevé des besoins des utilisateurs et de l'environnement sur lequel le logiciel fonctionne :



- *Gros Logiciels (volumineux)* - Il est plus facile de construire un mur que dans une maison ou un bâtiment, de même que la taille des logiciels devient importante.
- *Évolutivité* - Si le processus logiciel n'était pas basé sur des concepts scientifiques et techniques, il serait plus facile de recréer de nouveaux logiciels que de mettre à niveau un logiciel existant.
- *Prix* – comme L'industrie du matériel a montré ses compétences et une production importante a fait baisser le prix du matériel informatique électronique. Mais le coût du logiciel reste élevé si le processus approprié n'est pas adapté.
- *Nature dynamique* - La nature toujours croissante et adaptative du logiciel dépend énormément de l'environnement dans lequel l'utilisateur travaille. Si la nature du logiciel évolue constamment, de nouvelles améliorations doivent être apportées dans le logiciel existant. C'est là que l'ingénierie logicielle joue un rôle important.
- *Gestion de la qualité* - Un meilleur processus de développement logiciel fournit un produit logiciel de meilleure qualité.

I.5. CARACTERISTIQUES D'UN BON LOGICIEL

La caractéristique d'un logiciel est un ensemble de traits dominants ou l'expression de la correspondance entre une cause (*grandeur d'entrée*) et un effet (*grandeur de sortie*) dans la production ou le processus de développement des logiciels. Un produit logiciel doit évaluer en fonction de ce qu'il offre et de sa facilité d'utilisation. Un bon logiciel doit satisfaire les 3 catégories de critères suivants :

- Critères généraux ;
- Critères externes,
- Critères internes.

I.5.1. CRITERES GENERAUX

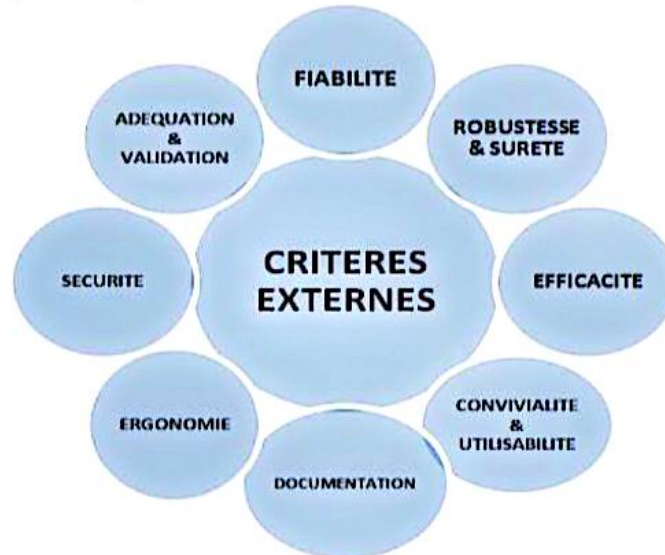
Les critères généraux sont en somme, des principes et éléments de référence qui permettent de juger ; d'estimer et de vérifier régulièrement si le processus de développement d'un logiciel possède ou non les différentes propriétés déterminées. Cette catégorie peut se matérialiser selon 3 différents processus (aspects) :

- *Opérationnel* : Cela nous indique dans quelle mesure le logiciel fonctionne bien dans les opérations. Ces opérations peuvent être mesurées entre autre part : *budgétisation, facilité d'utilisation, efficacité, exactitude, fonctionnalité, fiabilité, sécurité.*
- *Transitionnel* : Cet aspect est important lorsque le logiciel est déplacé d'une plate-forme à une autre : *Portabilité, Interopérabilité, Réutilisation et Adaptabilité.*
- *Maintenance* : Cet aspect explique comment un logiciel a la capacité de se maintenir dans une infrastructure et environnement en constante évolution : *maintenabilité, modularité, Flexibilité et Évolutivité.*

En résumé, le génie logiciel est une branche de l'informatique, qui utilise des concepts d'ingénierie bien définis pour produire des *produits logiciels efficaces, durables, évolutifs, économiques et ponctuels.*

I.5.2. CRITERES EXTERNES

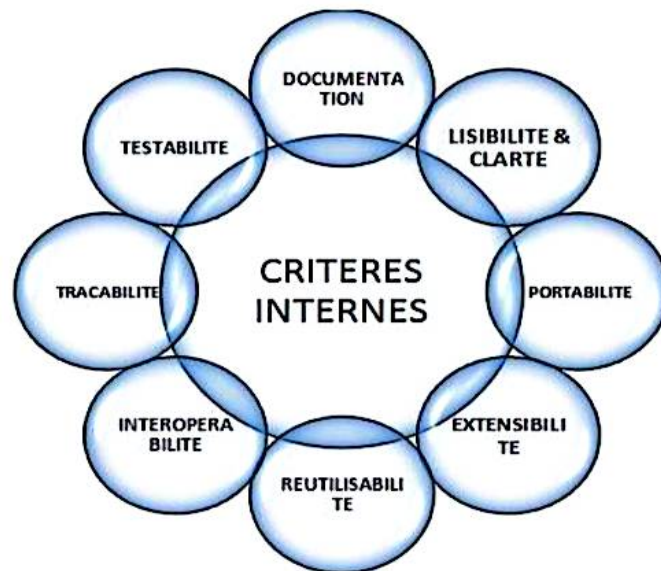
Les critères externes expriment ce qu'est un bon logiciel du point de vue des utilisateurs. Un logiciel de qualité doit être :



- **Fiabilité** : (*correction, justesse et conformité*) : le logiciel est conforme à ses spécifications, les résultats sont ceux attendus.
- **Robustesse et Sureté** : (*dysfonctionnements ou ne plante pas*) : le logiciel fonctionne raisonnablement en toutes circonstances, rien de catastrophique ne peut survenir, même en dehors des conditions d'utilisation prévues
- **Efficacité** : (*Le logiciel fait-il bon usage de ses ressources, en terme d'espace mémoire, et temps d'exécution*),
- **Convivialité et Utilisabilité** : (*Est-il facile et agréable à utiliser*),
- **Documentable** : (*accompagné d'un manuel utilisateur, ou d'un tutoriel*).
- **Ergonomique**⁵ : L'architecture du logiciel doit particulièrement être adaptée aux conditions de travail de l'utilisateur
- **Sécurité** : c'est la sûreté (*assurance*) et la garantie offerte par un logiciel, ou l'absence du danger lors de l'exploitation du logiciel.
- **Adéquation et validité** : c'est la conformité au maquetage du logiciel et au but qu'on se propose.
- **Intégrité** : c'est l'état d'un logiciel a conservé sans altération ses qualités et son degré originel. Autrement dit, C'est l'aptitude d'un logiciel à protéger son code et ses données contre des accès non autorisé.

I.5.3. CRITERES INTERNES

Les critères de qualité internes expriment ce qu'est un bon logiciel du point de vue du développeur. Ces critères sont essentiellement liés à la maintenance d'un logiciel. Un bon logiciel doit être facile à maintenir, et pour cela doit être :



- **Documentable :** *(a-t-il été précédé par un document de conception ou architecture).*
- **Lisibilité et Clarté :** *(est-il écrit proprement, et en respectant les conventions de programmation),*
- **Portabilité:** Un même logiciel doit pouvoir fonctionner sur plusieurs machines ainsi le rendre indépendant de son environnement d'exécution ;
- **Extensibilité :** *(est-il souple ou flexible ? ou permet-il l'ajout possible de nouvelles fonctionnalités).*
- **Réutilisabilité :** *(des parties peuvent être réutilisées pour développer d'autres logiciels similaires).*
- **Interopérabilité et coulabilité :** Un logiciel doit pouvoir interagir en synergie avec d'autres logiciels.
- **Traçabilité :** c'est la possibilité de suivre un produit aux différents stades de sa production, de sa transformation et de sa commercialisation.
- **Testabilité et vérifiabilité :** c'est la possibilité de soumettre un logiciel à une épreuve de confirmation de la tâche à exécuter.

I.6. CATEGORIES DE LOGICIELS

Il existe 2 catégories des logiciels notamment :

- **LOGICIELS GENERIQUES** : c'est sont des logiciels qui vendus comme les produits courants sur le marché informatique. Dans cette catégorie, on en distingue autant :

→ **Logiciels amateurs** : Il s'agit de logiciels développés par des « amateurs⁶ » (par exemple par des gens passionnés ou des étudiants qui apprennent à programmer). Bref, ce sont des logiciels sans impact économique significatif sur l'ensemble.

→ **Logiciels « jetables » ou « consommables »** : Il s'agit de logiciels comme par exemple les logiciels des traitements de texte ou les tableurs pour les entreprises. Ces logiciels ne coûtent pas très cher, et peuvent être remplacés facilement au sein d'une entreprise sans engendrer des risques majeurs. Ils sont souvent largement diffusés.

- **LOGICIELS SPECIFIQUES** : ce sont des logiciels développés pour une application précise et destinés à un seul client. Dans cette catégorie, on en distingue autant :

➤ **Logiciels essentiels au fonctionnement d'une entreprise** : Ce type de logiciel est le fruit d'un investissement non négligeable et doit avoir un comportement fiable, sûr et sécurisé.

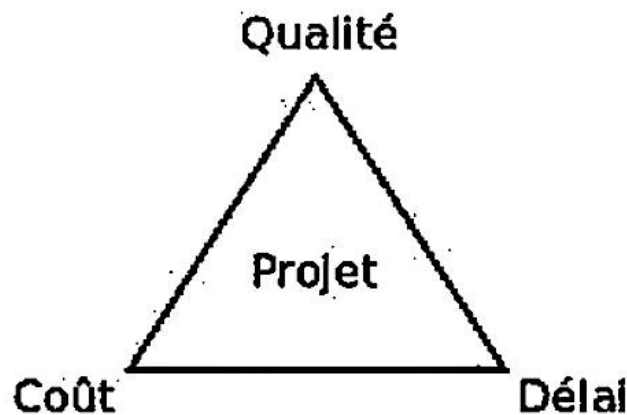
➤ **Logiciels critiques** : Il s'agit de logiciels dont l'exécution est vitale, pour lesquels une erreur peut coûter très cher ou coûter des vies humaines. Exemple : domaines du transport, de l'aviation, de la médecine, de l'armement, etc.

L'objectif de qualité d'un logiciel est différent suivant la catégorie de logiciel. En particulier, les logiciels essentiels au fonctionnement d'une entreprise, et plus encore les logiciels critiques, doivent avoir un haut niveau de qualité.

I.7. ENJEUX DU GENIE LOGICIEL

Un enjeu peut être considéré comme un ensemble des risques encourus par un développeur pour en mise en œuvre d'un logiciel. Le génie logiciel vise à rationaliser et à optimiser le processus de production d'un logiciel. Les enjeux associés sont multiples :

- Adéquation aux besoins du client.
- Respect des délais de réalisation prévus.
- Maximisation des performances et de la fiabilité.
- Facilitation de la maintenance et des évolutions ultérieures.



Comme tout projet, la réalisation d'un logiciel est soumise à des exigences contradictoires et difficilement conciliables (*triangle coût-délai-qualité*). La qualité d'un logiciel peut s'évaluer à partir d'un ensemble de facteurs tels que :

- Le logiciel répond-il aux besoins exprimés ?
- Le logiciel demande-t-il peu d'efforts pour évoluer aux regards de nouveaux besoins ?
- Le logiciel peut-il facilement être transféré d'une plate-forme à une autre ? ...

Sans être une solution miracle, le génie logiciel a pour objectif de maximiser la surface du triangle en tenant compte des priorités du client.

I.8. DIMENSIONS DU GENIE LOGICIEL

La dimension peut être envisagée comme étant un ensemble de mesures de chacune des grandeurs nécessaires à l'évaluation du logiciel. Le génie logiciel couvre l'ensemble du cycle de vie d'un logiciel. Il étudie toutes les activités qui mènent d'un besoin à la livraison du logiciel, y compris dans ses versions successives, jusqu'à sa fin de vie. Les dimensions du génie logiciel sont donc multiples :

- Analyse des besoins du client.
- Définition de l'architecture du logiciel.
- Choix de conception.
- Règles et méthodes de production du code source.
- Gestion des versions.
- Test du logiciel.
- Documentation.
- Organisation de l'équipe et interactions avec le client.
- ...

I.9. LES PRINCIPES DU GENIE LOGICIEL

Un certain nombre de grands principes (de bon sens) se retrouvent dans toutes ces méthodes. En voici une liste proposée par Ghezzi⁷ :

1. **La rigueur** : Les principales sources de défaillances d'un logiciel sont d'origine humaine. À tout moment, il faut se questionner sur la validité de son action. Des outils de vérification accompagnant le développement peuvent aider à réduire les erreurs. Cette famille d'outils s'appelle (*CASE "Computer Aided Software Engineering"*). C'est par exemple: typeurs, générateurs de code, assistants de preuves, générateurs de tests, outil d'intégration continue, fuzzer, . . .
2. **La Généralisation** : regroupement d'un ensemble de fonctionnalités semblables en une fonctionnalité paramétrable (*généricité⁸ et héritage*)
3. **La Structuration** : c'est la manière de décomposer un logiciel (*utilisation d'une méthode bottom-up ou top-down*). c'est la décomposition des problèmes en sous-problèmes indépendants⁹. Outre, Il s'agit de *la Décorrélation*. les problèmes pour n'en traiter qu'un seul à la fois. Ou de *la Simplification*. les problèmes (*temporairement*) pour aborder leur complexité progressivement. C'est par exemple : le modèle TCP.

4. **La modularité¹⁰** : Il s'agit de partitionner le logiciel *en modules* qui ont une *cohérence interne (des invariants)* ; et possèdent une *interface ne divulguant* sur le contenu du module que ce qui est strictement nécessaire aux modules clients. L'évolution de l'interface est *indépendante* de celle de l'implémentation du module. Les choix d'implémentation sont *indépendants* de l'utilisation du module. Ce mécanisme s'appelle le camouflage de l'information (*information hiding*).
5. **L'abstraction¹¹** : Il s'agit de présenter des *concepts généraux* regroupant un certain nombre de cas particuliers et de raisonner sur ces concepts généraux plutôt que sur chacun des cas particuliers. Le fait de fixer *la bonne granularité de détails* permet de raisonner plus efficacement et de factoriser le travail en instanciant le raisonnement général sur chaque cas particulier. C'est par exemple : les classes abstraites dans les langages à objets, le polymorphisme de *Caml* et le *generics* de Java, les foncteurs de *Caml* et le *templates* de C++, . . .
6. **La construction incrémentale**: Un développement logiciel a plus de chances d'aboutir s'il suit un cheminement *incrémental¹² (baby-steps)*
7. **La Documentation** : correspond à la gestion des documents incluant leur *identification, acquisition, production, stockage et distribution*. Il est primordial de prévoir les évolutions possibles d'un logiciel pour que la maintenance soit la plus efficace possible. Pour cela, il faut s'assurer que les modifications à effectuer soient le plus locales possibles. Ces modifications ne devraient pas être *intrusives* car les modifications du produit existant remettent en cause ses précédentes validations. Concevoir un système suffisamment riche pour que l'on puisse le modifier incrémentalement est l'idéal.
8. **La Vérification** : c'est la détermination du respect des spécifications établies sur la base des besoins identifiés dans la phase précédente du cycle de vie.

DEUXIEME CHAPITRE – LE CYCLE DE VIE DU DEVELOPPEMENT LOGICIEL

II.1. DEFINITION

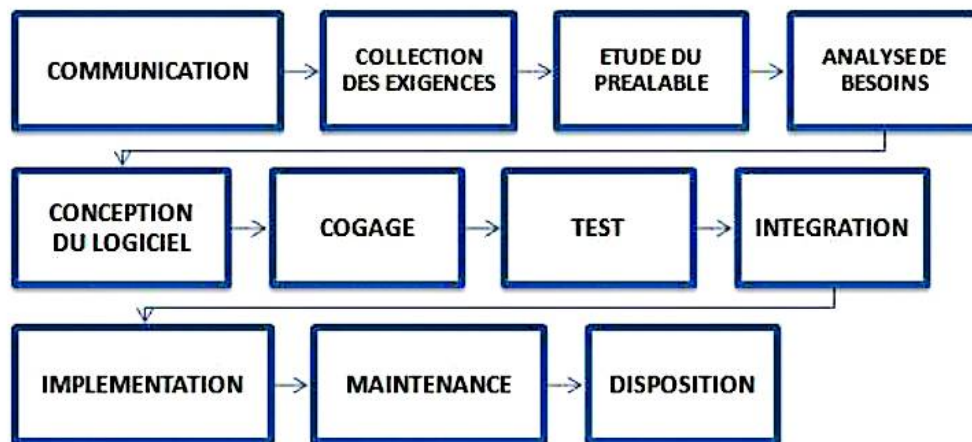
De par sa définition, Le Cycle de vie du développement d'un logiciel (SDLC¹³) est un ensemble d'étapes de la réalisation, de l'énoncé des besoins à la maintenance au retrait du logiciel sur le marché informatique. De façon générale, on peut dire que le cycle de vie du logiciel est la période de temps s'étalant du début à la fin du processus du logiciel. Il commence donc avec la proposition ou la décision de développer un logiciel et se termine avec sa mise hors service.

L'origine de ce découpage provient du constat que les erreurs ont un coût d'autant plus élevé qu'elles sont détectées tardivement dans le processus de réalisation. C'est ainsi que l'objectif principal du cycle de vie du développement d'un logiciel est de permettre la détection des erreurs au plus tôt et par conséquent, maîtriser la qualité du produit, les délais de sa réalisation et les coûts associés.

II.2. ÉTAPES DU CYCLE DE VIE DU DEVELOPPEMENT D'UN LOGICIEL

Le développement de logiciel impose d'effectuer un certain nombre d'étapes. Il existe de nombreux modèles de cycle de vie du développement d'un logiciel, les plus courants comportent les phases suivantes :

- La communication ;
- La collection des exigences ;
- L'étude du préalable (faisabilité ou opportunité) ;
- Définition et analyse de besoins (Spécification) ;
- La conception du logiciel ;
- Le codage ;
- Les tests ;
- L'intégration ;
- L'installation ;
- La maintenance ;
- La disposition.



II.1.1. LA COMMUNICATION

C'est la première étape du cycle de vie de développement logiciel où l'utilisateur initie la demande du produit logiciel souhaité. Ce dernier contacte le service du « *provider* » et essaie de négocier les termes du contrat. Il soumet alors sa demande à l'entreprise du fournisseur des services par écrit.

II.1.2. LA COLLECTION DES EXIGENCES

Cette étape fait avancer le travail de l'équipe du développement du logiciel jusqu'à la visibilité du projet informatique. A ce niveau, l'équipe de développement discute avec certains partenaires des divers problèmes du domaine et essaie de proposer autant d'informations possibles sur les différentes exigences. A ce niveau, les exigences sont contemplées et isolées en fonction des *besoins des utilisateurs* ; les *exigences du système* et les *exigences fonctionnelles*. Les exigences sont collectées en utilisant un nombre donné des pratiques telles que :

- Etude du système ou logiciel existant ou obsolète ;
- Conduite des interviews auprès des utilisateurs et développeurs ;
- Référencer aux différentes bases de données ;
- Et la collection des réponses au moyen de questionnaires.

II.1.3. ÉTUDE DU PREALABLE (*étude de l'opportunité*)

Le développement est précédé d'une étude d'opportunité ou étude préalable. Cette phase a comme objectif de répondre aux questions suivantes : Pourquoi développer le logiciel ? Comment procéder pour faire ce développement ? Quels moyens faut-il mettre en œuvre ?

Elle comprend à la fois des aspects techniques et de gestion. Parmi les tâches techniques, groupées sous le terme étude préalable, on peut citer :

- Dresser un état de l'existant et faire une analyse de ses forces et faiblesses ;
- Identifier les idées ou besoins de l'utilisateur ;
- Formuler des solutions potentielles ;
- Faire des études de faisabilité ;
- Planifier la transition entre l'ancien logiciel et le nouveau, s'il y a lieu ;
- Affiner ou finaliser l'énoncé des besoins de l'utilisateur.

II.1.4. DEFINITION ET ANALYSE DES BESOINS (*spécification*¹⁴)

Lors de la phase d'analyse, également appelée phase de spécification (*requirements phase, analysis phase, definition phase*), on analyse les besoins de l'utilisateur ou du système englobant et on définit ce que le logiciel devra faire. Le résultat de la phase d'analyse est consigné dans un document appelé « *cahier des charges du logiciel ou spécification du logiciel* », en anglais : « *software requirements, software specification ou requirements specification* ». Il est essentiel qu'une spécification ne définisse que les caractéristiques essentielles du logiciel pour laisser de la place aux décisions de conception (*Ne pas faire de choix d'implémentation à ce niveau*). Une spécification comporte les éléments suivants :

- description de l'environnement du logiciel ;
- spécification fonctionnelle (*functional specification*), qui définit toutes les fonctions que le logiciel doit offrir ;
- comportement en cas d'erreurs, c'est-à-dire dans les cas où le logiciel ne peut pas accomplir une fonction ;
- performances requises (*performance requirements*), par exemple : temps de réponse, encombrement en mémoire, sécurité de fonctionnement ;
- les interfaces avec l'utilisateur (*user interface*), en particulier le dialogue sur terminal, la présentation des écrans, la disposition des états imprimés, etc.
- interfaces avec d'autres logiciels et le matériel ;
- contraintes de réalisation, telles que l'environnement de développement, le langage de programmation à utiliser, les procédures et normes à suivre, etc.

Il est judicieux de préparer pendant la phase d'analyse les procédures qui seront mises en œuvre pour vérifier que le logiciel, une fois construit, est conforme à la spécification, que nous l'appellerons **test de réception** (*acceptance test*). Durant la phase d'analyse, on produit également une version provisoire des manuels d'utilisation et d'exploitation du logiciel. Les Points clés :

- Pour les gros systèmes, il est difficile de formuler une spécification définitive. C'est pourquoi on supposera que les besoins initiaux du système sont incomplets et inconsistants.
- La définition des besoins et la spécification des besoins constituent des moyens de description à différents niveaux de détails, s'adressant à différents lecteurs.
- La définition des besoins est un énoncé, en langue naturelle, des services que le système est sensé fournir à l'utilisateur. Il doit être écrit de manière à être compréhensible par les décideurs côté client et côté contractant, ainsi que par les utilisateurs et acheteurs potentiels du système.
- La spécification des besoins est un document structuré qui énonce les services de manière plus détaillée. Ce document doit être suffisamment précis pour servir de base contractuelle entre le client et le fournisseur du logiciel. On peut utiliser des techniques de spécification formelle pour rédiger un tel document, mais cela dépendra du bagage technique du client.
- Il est difficile de détecter les inconsistances ou l'incomplétude d'une spécification lorsqu'elle est décrite dans un langage naturel non structuré. On doit toujours imposer une structuration du langage lors de la définition des besoins.
- Les besoins changent inévitablement. Le cahier des charges doit donc être conçu de manière à être facilement modifiable

II.1.5. LA CONCEPTION DU LOGICIEL

La phase d'analyse est suivie de la phase de conception, généralement décomposée en deux phases successives :

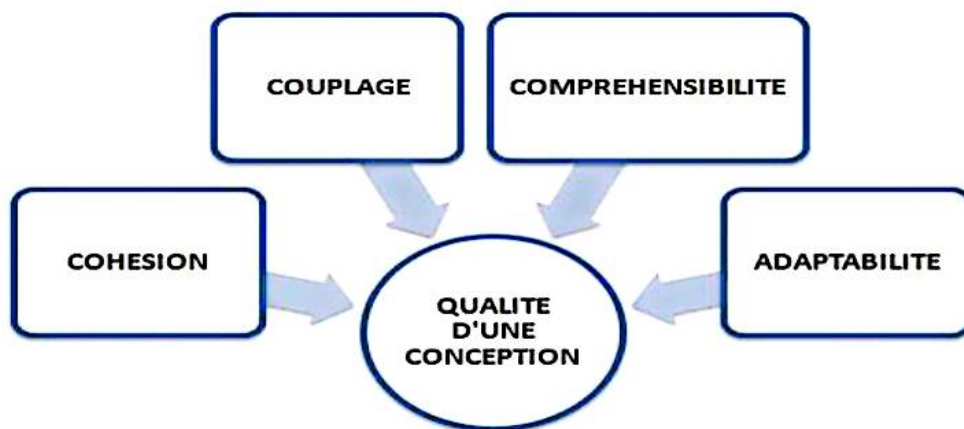
- **Conception générale ou conception architecturale (*preliminary design ou architectural design*)** : Si nécessaire, il faut commencer par l'ébauche de plusieurs variantes de solutions et choisir celle qui offre le meilleur rapport entre coûts et avantages. Il faut ensuite figer la solution retenue, la décrire et la détailler. En particulier, il faut décrire l'architecture de la solution, c'est-à-dire son organisation en entités, les interfaces de ces entités et les interactions entre ces entités. Ce

processus de structuration doit être poursuivi jusqu'à ce que tous les éléments du document de spécification aient été pris en compte. Le résultat de cette démarche est « **un document de conception générale** ». Durant la phase de conception générale, il faut également préparer **la phase d'intégration**. A cet effet, il faut élaborer **un plan d'intégration**, y compris **un plan de test d'intégration**.

- **Conception détaillée (*detailed design*)**: La conception détaillée affine la conception générale. Elle commence par décomposer les entités découvertes lors de la conception générale en entités plus élémentaires. Cette décomposition doit être poursuivie jusqu'au niveau où les entités sont faciles à implémenter et à tester, c'est-à-dire correspondent à des composants logiciels élémentaires. Ce niveau dépend fortement du langage de programmation retenu pour l'implémentation. Il faut ensuite décrire chaque composant logiciel en détail : son interface, les algorithmes utilisés, le traitement des erreurs, ses performances, etc. L'ensemble de ces descriptions constitue le document de conception détaillée. Pendant la conception détaillée, il faut également préparer la vérification des composants logiciels élémentaires qui fera l'objet de la phase des tests unitaires. Le résultat est consigné dans un document appelé plan de tests unitaires. Si nécessaire, il faut de plus compléter le plan d'intégration, car de nouvelles entités ont pu être introduites pendant la conception détaillée.

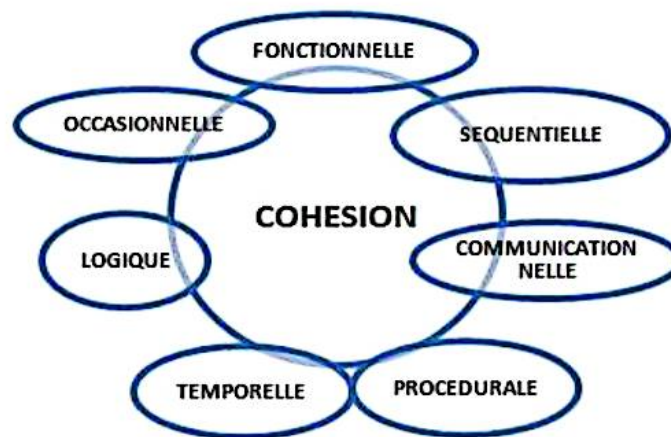
II.1.5.1. QUALITE D'UNE CONCEPTION LOGICIELLE

La composante la plus importante de la qualité d'une conception est la maintenabilité. C'est en maximisant la cohésion à l'intérieur des composants et en minimisant le couplage entre ces composants que l'on parviendra à une conception maintenable :



A. La Cohésion

La cohésion d'un composant permet de mesurer la qualité de sa structuration, autrement dit, c'est un ensemble des forces solidaires développées à l'intérieur d'un logiciel. Un composant devrait implémenter une seule fonction logique ou une seule entité logique. La cohésion est une caractéristique désirable car elle signifie que chaque unité ne représente qu'une partie de la résolution du problème. *Constantine et Yourdon* identifient, en 1979, sept niveaux de cohésion présentés ci-après, du plus fort au plus faible :



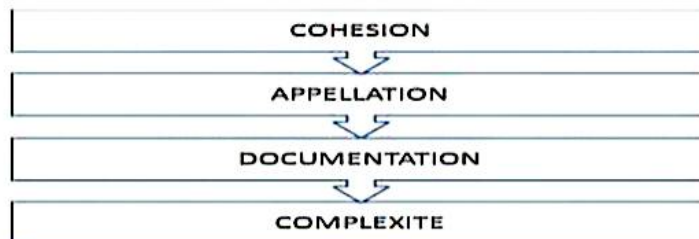
- **La cohésion fonctionnelle** : Ici, Le module assure une seule fonction et Tous les éléments du composant contribuent à atteindre un seul objectif (si *un élément est supprimé, l'objectif ne sera pas atteint*).
- **La cohésion séquentielle** : Dans ce type de cohésion, la sortie d'un élément est utilisée en entrée d'un autre (*dans ce cas, l'ordre des actions est important*).
- **La cohésion de communication** : Lorsque tous les éléments d'un composant travaillent sur les mêmes données.
- **La cohésion procédurale** : Dans ce cas, les éléments d'un composant constituent une seule séquence de contrôle.
- **La cohésion temporelle** : On parle de cohésion temporelle quand dans un même composant sont regroupés tous les éléments qui sont activés au même moment, par exemple, à l'initialisation d'un programme ou encore en fin d'exécution.
- **La cohésion logique** : Tous les éléments d'un composant effectuent des opérations semblables comme, par exemple, module qui édite tous les types de transactions et difficile à modifier.
- **La cohésion occasionnelle** : Le découpage en modules conduit à ce qu'une fonction se retrouve assurée par plusieurs modules. Dans ce cas, il n'y a pas de relation entre les éléments du composant.

B. Le Couplage

C'est la situation de deux ou plusieurs états dont les impératifs de défense sont assurés par l'intégration de leurs forces respectives dans le cadre d'une stratégie commune. Le couplage est relatif à la cohésion. C'est une indication de la force d'interconnexion des différents composants d'un système. En règle générale, des modules sont fortement couplés lorsqu'ils utilisent des variables partagées ou lorsqu'ils échangent des informations de contrôle.

C. La compréhensibilité

Pour modifier un composant dans une conception, il faut que celui qui est responsable de cette modification comprenne l'opération effectuée par ce composant. Cette compréhensibilité dépend des caractéristiques :



- **La cohésion** : Le composant peut-il être compris sans que l'on fasse référence à d'autres composants?
- **L'appellation** : Les noms utilisés dans le composant sont-ils significatifs ? Des noms significatifs reflètent les noms des entités du monde réel que l'on modélise.
- **La documentation** : Le composant est-il documenté de manière à ce que l'on puisse établir une correspondance claire entre le monde réel et le composant ? Est-ce que cette correspondance est résumée quelque part.
- **La complexité** : Les algorithmes utilisés pour implémenter le composant sont-ils complexes ?

D. L'adaptabilité

Si l'on doit maintenir une conception, cette dernière doit être facilement adaptable. Bien sûr, il faut pour cela que les composants soient faiblement couplés. En plus de ça, la conception doit être bien documentée, la documentation des composants doit être facilement compréhensible et consistante avec l'implémentation, cette dernière devant elle aussi être écrite de manière lisible.

II.1.6. LE CODAGE

Après la conception détaillée, on peut passer à la phase de codage, également appelée phase de construction, phase de réalisation ou phase d'implémentation (*implémentation phase, construction phase, coding phase*). Lors de cette phase, la conception détaillée est traduite dans un langage de programmation.

II.1.7. TESTS

La phase d'implémentation est suivie de la phase de test (*test phase*). Durant cette phase, les composants du logiciel sont évalués et intégrés, et le logiciel lui-même est évalué pour déterminer s'il satisfait la spécification élaborée lors de la phase d'analyse. Cette phase est en général subdivisée en plusieurs phases.

Lors des tests unitaires (*unit test*), on évalue chaque composant individuellement pour s'assurer qu'il est conforme à la conception détaillée. Si ce n'est déjà fait, il faut élaborer pour chaque composant un jeu de données de tests. Il faut ensuite exécuter le composant avec ce jeu, comparer les résultats obtenus aux résultats attendus, et consigner le tout dans le document des tests unitaires. S'il s'avère qu'un composant comporte des erreurs, il est renvoyé à son auteur, qui devra diagnostiquer la cause de l'erreur puis corriger le composant. Le test unitaire de ce composant est alors à reprendre.

II.1.8. INTEGRATION

Après avoir effectué avec succès la phase des tests de tous les composants, on peut procéder à leur assemblage, qui est effectué pendant la phase d'intégration (*intégration phase*). Pendant cette phase, on vérifie également la bonne facture des composants assemblés, ce qu'on appelle le test d'intégration (*intégration test*). On peut donc distinguer les actions suivantes :

- construire par assemblage un composant à partir de composants plus petits ;
- exécuter les tests pour le composant assemblé et enregistrer les résultats ;
- comparer les résultats obtenus aux résultats attendus ;
- si le composant n'est pas conforme, engager la procédure de modification ;
- si le composant est conforme, rédiger les comptes-rendus du test d'intégration et archiver sur support informatique les sources, objets compilés, images exécutables, les jeux de tests et leurs résultats.

II.1.9. INSTALLATION

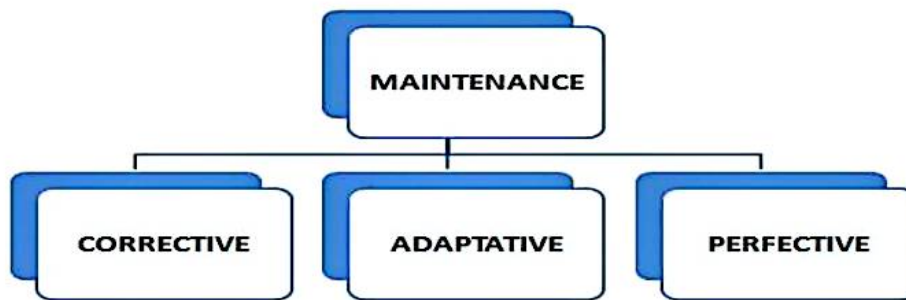
Après avoir intégré le logiciel, on peut l'installer dans son environnement d'exploitation, ou dans un environnement qui simule cet environnement d'exploitation, et le tester pour s'assurer qu'il se comporte comme requis dans la spécification élaborée lors de la phase d'analyse.

II.1.10. MAINTENANCE

Après l'installation suit la phase d'exploitation et de maintenance (*operation and maintenance phase*). Le logiciel est maintenant employé dans son environnement opérationnel, son comportement est surveillé et, si nécessaire, il est modifié. Cette dernière activité s'appelle la maintenance du logiciel (*software maintenance*).

Il peut être nécessaire de modifier le logiciel pour corriger des défauts, pour améliorer ses performances ou autres caractéristiques, pour adapter le logiciel à un nouvel environnement ou pour répondre à des nouveaux besoins ou à des besoins modifiés. On peut donc distinguer entre la maintenance corrective, la maintenance perfective et la maintenance adaptative. Sauf pour des corrections mineures, du genre dépannage, la maintenance exige en fait que le cycle de développement soit réappliqué, en général sous une forme simplifiée. Voici les différentes formes de maintenance qui peuvent exister en génie logiciel :

- **La Maintenance corrective** : c'est une maintenance qui Corrige les erreurs et les défauts d'utilité, d'utilisabilité, de fiabilité... cette maintenance Identifie également les défaillances, et les dysfonctionnements en localisant la partie du code responsable. Elle Corrige et estime l'impact d'une modification. Attention, La plupart des corrections introduisent de nouvelles erreurs et Les coûts de correction augmentent exponentiellement avec le délai de détection. Bref, la maintenance corrective donne lieu à de nouvelles livraisons (*release*).
- **La Maintenance adaptative** : c'est une maintenance qui ajuste le logiciel pour qu'il continue à remplir son rôle compte tenu du l'évolution des Environnements d'exécution, des Fonctions à satisfaire et des Conditions d'utilisation. C'est par exemple le changement de SGBD, de machine, de taux de TVA, an 2000 ...
- **La Maintenance perfective et d'extension** : c'est une maintenance qui sert a accroître et améliorer les possibilités du logiciel afin de donne lieu à de nouvelles versions. C'est par exemple ; les services offerts, l'interface utilisateur, les performances...



Une fois qu'une version modifiée du logiciel a été développée, il faut bien entendu la distribuer. De plus, il est en général nécessaire de fournir à l'exploitant du logiciel une assistance technique et un support de consultation. En résumé, on peut dire que la maintenance et le support du logiciel comprennent les tâches suivantes :

- effectuer des dépannages pour des corrections mineures ;
- réappliquer le cycle de développement pour des modifications plus importantes ;
- distribuer les mises à jour ;
- fournir l'assistance technique et un support de consultation ;
- maintenir un journal des demandes d'assistance et de support.

A un moment donné, on décide de mettre le logiciel hors service. Les tâches correspondantes sont accomplies durant la phase de retrait (*retirement phase*) et comprennent :

- avertir les utilisateurs ;
- effectuer une exploitation en parallèle du logiciel à retirer et de son successeur ;
- arrêter le support du logiciel.

Ces étapes ne doivent pas être vues comme se succédant les unes aux autres de façon linéaire. Il y a en général (*et toujours*) des retours sur les phases précédentes, en particulier si les tests ne réussissent pas ou si les besoins évoluent.

II.1.11. DISPOSITION

Comme en informatique, le temps est un facteur indispensable qui doit être surveillé quant à tout ce qui concerne le développement des produits informatiques. C'est ainsi qu'un logiciel peut subir une détérioration proportionnellement à sa performance. Il peut être complètement obsolète ou exiger une intense mise à jour. Cette phase inclut l'archivage des données et les exigences de composants logiciels ; la fermeture du système ; la disposition planifiée des activités et de la terminaison du système quant à la période appropriée de la terminaison du logiciel. Bref, cette phase concerne la gestion de versions.

TROISIEME CHAPITRE - MODELISATION DE PROCESSUS DE DEVELOPPEMENT LOGICIEL

La modélisation de développement logiciel aide les développeurs à sélectionner la stratégie pour développer le produit logiciel. Ainsi, chaque modélisation de développement possède ses outils propres, ses méthodes et ses procédures qui permettent d'exprimer clairement et définissent le cycle de vie de développement du logiciel.

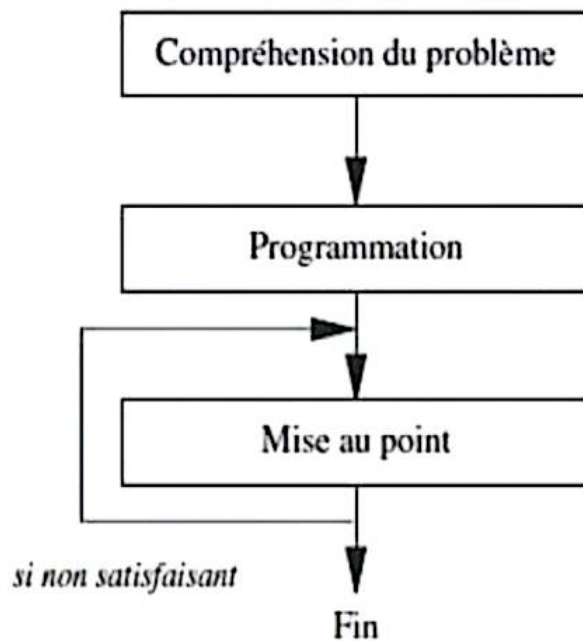
Les modèles du cycle de vie du logiciel sont des « *plans de travail* » qui permettent de planifier le développement. Plus le logiciel à développer est complexe (*taille, algorithmes*) et critique, plus il est important de bien contrôler le processus de développement et plus les documents qui accompagnent le logiciel doivent être précis et détaillés.

Il s'avère de nos jours, que nous ne pouvons plus avoir une démarche unique dans le développement de projets informatiques, mais qu'il faut construire le découpage temporel en fonction des caractéristiques de l'entreprise et du projet. On s'appuie pour cela sur des découpages temporels génériques, appelés *modèles de développement* (*process models*) ou modèles de cycle de vie d'un projet informatique. Les principaux modèles sont :

- le modèle du *code-and-fix* ;
- le modèle de la transformation automatique ;
- le modèle de la cascade ;
- le modèle en V ;
- le modèle par incrément;
- le modèle par prototypage ;
- le modèle de la spirale.

III.1. LE MODELE DU CODE-AND-FIX

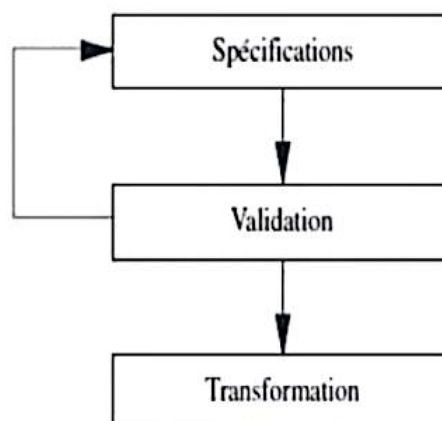
C'est un modèle qui repose sur la possibilité d'une détermination facile des besoins : une première phase de Compréhension du problème est suivie d'une phase de Programmation ; puis une phase de Mise au point, parfois en collaboration avec l'utilisateur du futur système, est répétée jusqu'à l'atteinte du résultat visé.



Le modèle du code-and-fix.

II.2. LE MODELE DE TRANSFORMATION AUTOMATIQUE

C'est un modèle basé sur la possibilité de transformer automatiquement des spécifications en programmes. L'essentiel de l'effort va donc porter sur une description exhaustive des spécifications qui devront être complètement validées. Une succession de cycles *Phase de Spécifications* – *Phase de Validation* s'achève par la *phase de Transformation*, où s'effectue la génération du code.



Le modèle de la transformation automatique